Laboratory Exercise 3

Latches, Flip-flops, and Registers

The purpose of this exercise is to investigate latches, flip-flops, and registers.

Part I

Intel FPGAs include flip-flops that are available for implementing a user's circuit. We will show how to make use of these flip-flops in Part II of this exercise. But first we will show how storage elements can be created in an FPGA without using its dedicated flip-flops.

Figure 1 depicts a gated RS latch circuit. A style of VHDL code that uses logic expressions to describe this circuit is given in Figure 2. If this latch is implemented in an FPGA that has 4-input lookup tables (LUTs), then only one lookup table is needed, as shown in Figure 3a.

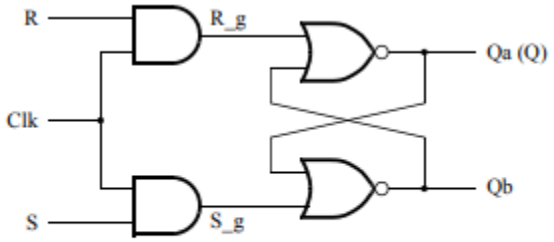


Figure 1: A gated RS latch circuit.

- A gated RS latch described the hard way

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

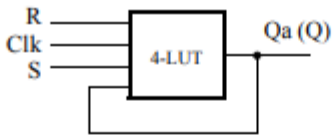
ENTITY part1 IS
    PORT ( Clk, R, S : IN      STD_LOGIC;
                Q          : OUT    STD_LOGIC);
END part1;

ARCHITECTURE Structural OF part1 IS
    SIGNAL R_g, S_g, Qa, Qb : STD_LOGIC ;
    ATTRIBUTE KEEP : BOOLEAN;
    ATTRIBUTE KEEP OF R_g, S_g, Qa, Qb : SIGNAL IS TRUE;
BEGIN
    R_g <= R AND Clk;
    S_g <= S AND Clk;
    Qa <= NOT (R_g OR Qb);
    Qb <= NOT (S_g OR Qa);
```

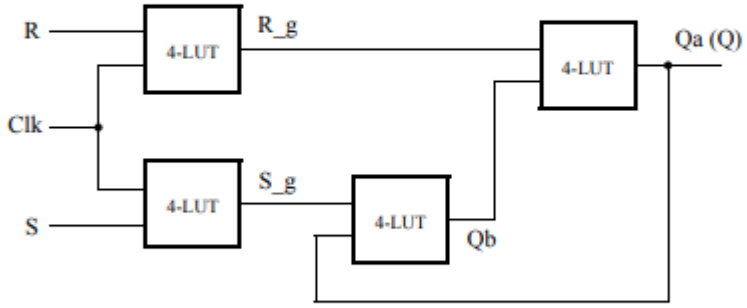
```
    Q <= Qa;
END Structural;
```

Figure 2: Specifying the RS latch by using logic expressions.

Although the latch can be correctly realized in one 4-input LUT, this implementation does not allow its internal signals, such as R_g and S_g, to be observed, because they are not provided as outputs from the LUT. To preserve these internal signals in the implemented circuit, it is necessary to include a compiler directive in the code. In Figure 2 the directive KEEP is included by using a VHDL ATTRIBUTE statement to instruct the Quartus compiler to use separate logic elements for each of the signals R_g; S_g; Qa; and Qb. Compiling the code produces the circuit with four 4-LUTs depicted in Figure 3b.



(a) Using one 4-input lookup table for the RS latch.



(b) Using four 4-input lookup tables for the RS latch.

Figure 3: Implementation of the RS latch from Figure 1.

Create a Quartus project for the RS latch circuit as follows:

1. Create a new Quartus project for your DE1-series board.
2. Generate a VHDL file with the code in Figure 2 and include it in the project.
3. Compile the code. Use the Quartus RTL Viewer tool to examine the gate-level circuit produced from the code, and use the Technology Map Viewer tool to verify that the latch is implemented as shown in Figure 3b.

4. Simulate the behavior of your VHDL code by using the simulation feature provided in the Quartus software. First, create a vector waveform file (*.vwf) using the Quartus software to specify the inputs and outputs of your circuit. Then, use the commands available in the Quartus Simulation Waveform Editor tool to run a simulation of the circuit. The procedure for using the Simulation Waveform Editor to perform a simulation is described in the tutorial Quartus Introduction, available on the Intel FPGA University Program website (more detailed instructions are provided in the tutorial Introduction to Quartus Simulation). An example of a vector waveform file is displayed in Figure 4. The waveforms in the figure begin by setting Clk = 1 and R = 1, which allows the simulation tool to initialize all of the signals inside of the latch to known values. If the waveforms provided to the simulation tool do not allow for initialization of all signals in the circuit, then the simulation tool may exit with an error condition.

5. Verify that the latch works properly for all input conditions by using functional simulation.

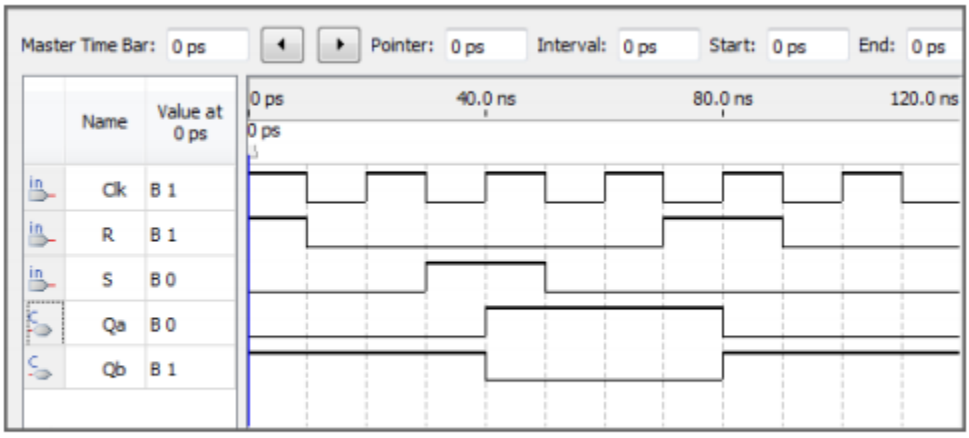


Figure 4: Simulation waveforms for the RS latch.

Part II

Figure 5 shows the circuit for a gated D latch.

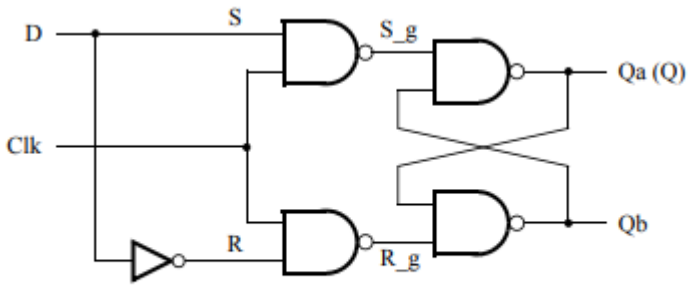
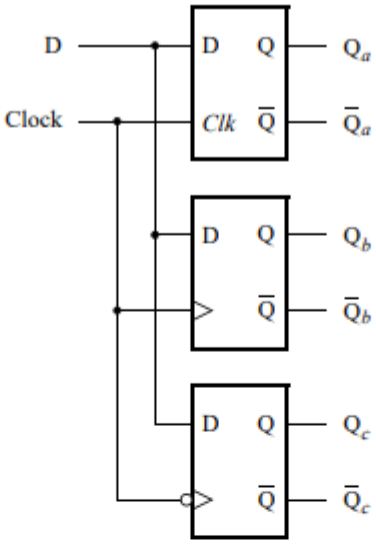
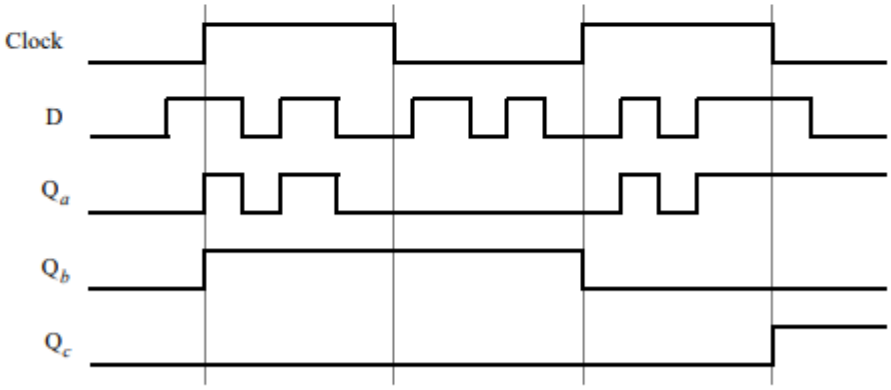


Figure 5: Circuit for a gated D latch.

Figure 7 shows a circuit with three different storage elements: a gated D latch, a positive-edge triggered D flip-flop, and a negative-edge triggered D flip-flop.



(a) Circuit



(b) Timing diagram

Figure 7: Circuit and waveforms for Part IV.

Implement and simulate this circuit using the Quartus software as follows:

1. Create a new Quartus project.

2. Write a VHDL file that instantiates the three storage elements. For this part you should no longer use the KEEP directive (that is, the VHDL ATTRIBUTE statement) from Part I. Figure 8 gives a behavioral style of VHDL code that specifies the gated D latch in Figure 5. This latch can be implemented in one 4-input lookup table. Use a similar style of code to specify the flip-flops in Figure 7.

3. Compile your code and use the Technology Map Viewer to examine the implemented circuit. Verify that the latch uses one lookup table and that the flip-flops are implemented using the flip-flops provided in the target FPGA.

4. Create a Vector Waveform File (.vwf) that specifies the inputs and outputs of the circuit. Draw the inputs D and Clock as indicated in Figure 7. Use functional simulation to obtain the three output signals. Observe the different behavior of the three storage elements.

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY latch IS
   PORT ( D, Clk : IN      STD_LOGIC ;
                Q        : OUT STD_LOGIC) ;
END latch ;

ARCHITECTURE Behavior OF latch IS
BEGIN
   PROCESS ( D, Clk )
   BEGIN
      IF Clk = '1' THEN
         Q<=D;
      END IF ;
   END PROCESS ;
END Behavior ;
```

Figure 8: A behavioral style of VHDL code that specifies a gated D latch.

Part III

We wish to display the hexadecimal value of an 8-bit number A on the two 7-segment displays HEX3 2. We also wish to display the hex value of an 8-bit number B on the two 7-segment displays HEX1 - 0. The values of A and B are inputs to the circuit which are provided by means of switches SW_{7-0}. To input the values of A and B, first set the switches to the desired value of A, store these switch values in a register, and then change the switches to the desired value of B. Finally, use an adder to generate the arithmetic sum S = A + B, and display this sum on the 7-segment displays HEX5 - 4. Show the carry-out produced by the adder on LEDR(0).

1. Create a new Quartus project which will be used to implement the desired circuit on your DE1-series board.
2. Write a VHDL file that provides the necessary functionality. Use KEY_0 as an active-low asynchronous reset, and use KEY_1 as a clock input.
3. Include the necessary pin assignments for the pushbutton switches and 7-segment displays, and then compile the circuit.
4. Download the circuit onto your DE1-series board and test its functionality by toggling the switches and observing the output displays.